PREDICTIVE MAINTENANCE SYSTEM

Project Journey, Prompt History, Design Decisions & Limitations

Project record

Prepared from the discussions and implementation decisions made during 10–12 November 2026.

Purpose of this document

This document records how the project was shaped through discussion, decisions and step-by-step implementation. It distinguishes the project choices and work directed by the developer from the specific areas where ChatGPT was used as a planning, explanation, coding and troubleshooting assistant.

Important accuracy note: the prompts below are faithful summaries of the requests and decisions visible in the conversation history, not a claim that every line is a verbatim transcript. The document does not imply that the project has been validated in a real factory environment.

1. Project overview

The project is a web-based Predictive Maintenance System intended to monitor machine and sensor telemetry, identify abnormal operating behaviour, estimate the risk of machine failure and provide maintenance-oriented information such as Remaining Useful Life (RUL). The agreed implementation uses a Python backend, MySQL database, machine-learning components, a browser frontend built with HTML/CSS/Vanilla JavaScript, and Server-Sent Events (SSE) for live updates.

Agreed project configuration

Area	Decision / scope
Failure prediction horizon	Predict whether a machine is likely to fail within the next 24 hours.
Periodic analysis	Run scheduled analysis every 10 minutes.
Prediction triggers	Use both scheduled/periodic prediction and event-driven prediction.
ML approach	Use both a failure-classification approach and anomaly-detection approach.
RUL	Include Remaining Useful Life as a project output/feature.
Backend	Python application using FastAPI in the proposed architecture.
Database	MySQL (selected instead of PostgreSQL).
Frontend	HTML, CSS and Vanilla JavaScript.
Live updates	Server-Sent Events (SSE).
Access control	Admin sign-in and role-based access control (RBAC).
Packaging boundary	Local project setup; Docker and deployment were explicitly removed from the requested scope.

2. Journey and prompt history

The sequence below captures the main questions, choices and requests that shaped the system. It is organised by project stage rather than presenting a fabricated word-for-word chat transcript.

Stage 1 — Define the problem and the system's purpose (10 November 2026)

Developer-led discussion: The project began with a Predictive Maintenance System problem statement. The initial focus was to understand what the system should monitor, what a useful prediction means, and how the system would support maintenance decisions.

Representative prompt/request: “Read the problem statement and help identify the next logical step for our Predictive Maintenance System.”

Discussion outcome: The project was framed as an end-to-end system rather than only a machine-learning notebook. The discussion covered telemetry, database storage, prediction outputs, alerts, traceability and a web interface.

Stage 2 — Decide how data reaches the ML models

Developer-led question: “Will the ML models take real-time data through the database, and will predictions run every minute?”

Follow-up: “How can we generate data per minute for feeding to the ML models?”

Discussion outcome: We separated model training from inference. Historical data can be used to train models, while newly generated or ingested telemetry is stored in the database and read by the inference process. A simulator can produce timestamped telemetry for development when physical sensors are not connected.

Important distinction: Generating simulated readings is a development method. It is not the same as collecting genuine live readings from industrial equipment.

Stage 3 — Select the prediction strategy and outputs (10 November 2026)

Developer decisions: You selected both real-time/event-triggered and periodic prediction, both ML approaches, and RUL. You then finalised a 24-hour failure prediction horizon and a 10-minute periodic analysis interval.

Representative prompts/decisions:

- "I choose prediction frequency: both."
- "ML approach: both."
- "Finalise failure prediction horizon = 24 hours; periodic analysis interval = 10 minutes; include RUL = yes."
- "Now start the implementation plan with database schema, ML dataset preparation and Python training scripts."

Discussion outcome: The agreed prediction target became whether a machine will fail within the next 24 hours. Periodic inference was set to every 10 minutes, with event-driven inference also included. RUL was retained as an additional maintenance-oriented output.

Stage 4 — Decide the user interface and access model (10 November 2026)

Developer questions: You asked whether HTML, CSS and Vanilla JavaScript could be used for the frontend, specified SSE, and asked how admin sign-in, user sign-up, role creation and authentication during subsequent sign-in would work.

Representative prompts:

- "Can we use HTML CSS Vanilla JS for frontend?"
- "No, we have to use SSE."
- "Is there an admin sign-in to access the website?"
- "How will sign-up for different roles be created and checked for authentication while signing in again?"

Discussion outcome: The frontend choice was HTML/CSS/Vanilla JavaScript, with SSE for server-to-browser streaming updates. Authentication and role-based access were included in the scope, with an administrator role and controlled access for other roles.

Stage 5 — Request the project structure and implementation package (10–11 November 2026)

Developer request: You asked for the complete project, executable code, requirements.txt, README and a project folder, then requested the project structure and a step-by-step build sequence.

Representative prompts:

- "Build the whole project and write all executable code and also provide requirements.txt and README."
- "Provide me project structure."
- "Guide me how to start building the project step by step. First give me step 1; after that next step."
- "How many steps are left?" and "How to stop the project; we will start tomorrow."

Discussion outcome: The work moved from system design into an ordered local implementation workflow. Rather than treating the generated project as finished, the implementation was approached in stages, with the developer running steps and reporting results before proceeding.

Stage 6 — Refine the technology and delivery boundary (11 November 2026)

Developer decisions: You explicitly asked to remove deployment and Docker, and to use MySQL for the database. You asked for the corrected project package after those changes.

Representative prompt: “Don't need deployment for project; remove Docker and use MySQL for database. Edit this and again give me the corrected project file.”

Discussion outcome: The agreed local-development setup used MySQL rather than PostgreSQL, and Docker/deployment were excluded. The main functional scope—authentication, simulator, ML failure prediction and RUL, scheduling, event-driven inference, APIs and SSE—was retained in the revised plan.

Stage 7 — Build and troubleshoot incrementally (11–12 November 2026)

Developer-led implementation: You proceeded through the implementation steps, executed commands and code in your own environment, and returned with progress updates such as “done” and “working”. When an issue appeared, you shared a screenshot and asked for help interpreting it before moving to the next step.

Representative prompts:

- “Done, next.”
- “It is showing this” (with a screenshot of the issue).
- “Done, working.”
- “I want to change admin email now.”
- “Give me complete code for frontend without testing, no patch code. I want complete code.”
- “Give a README explaining how to run it because I am going to upload it on GitHub.”

Discussion outcome: The process was iterative: you ran the project, checked the result, supplied feedback and chose when to continue. ChatGPT provided targeted explanations, implementation guidance, code and troubleshooting support when requested.

3. Who drove the work? — A fair account of tool usage

The project should be presented as developer-led work supported by an AI assistant, not as a project created entirely by ChatGPT. The conversation history shows that you made the key product and technical decisions and carried out the step-by-step implementation in your own environment.

Decisions and work directed by the developer

- Defined the project goal and kept the scope focused on predictive maintenance.
- Chose the 24-hour failure horizon, 10-minute periodic interval, dual prediction triggers, dual ML approach and RUL.
- Selected HTML/CSS/Vanilla JavaScript and required SSE for live updates.
- Asked for and shaped the authentication and role-based access requirements.
- Changed the database choice to MySQL and removed Docker/deployment from the requested scope.
- Executed implementation steps, checked outputs, reported errors and requested the next step only after completing the previous one.
- Requested specific changes such as updating the admin email and obtaining a complete frontend implementation and GitHub README.

Where ChatGPT was used

- Discussing architecture and helping translate requirements into an implementation sequence.
- Explaining database, telemetry ingestion, simulation, training and inference concepts.
- Providing code examples, project structure and implementation guidance for selected components.
- Helping diagnose issues based on the screenshots and results you supplied.
- Producing documentation and explaining how to run the project.

A credible way to describe the collaboration: “I designed the requirements and made the key architecture and technology decisions for the Predictive Maintenance System. I implemented the project step by step in my local environment, using ChatGPT selectively to discuss design choices, clarify concepts, generate or refine specific code sections, and troubleshoot issues when needed. I reviewed and ran the implementation rather than relying on AI to make project decisions independently.”

4. Project limitations and current boundaries

The following limitations should be stated honestly in a report, presentation or interview. They describe the current project scope and the risks that would need to be addressed before industrial deployment; they are not evidence that every item has already failed or been tested.

Data and sensor limitations

- **Simulated telemetry:** If the development dataset is generated by a simulator, it may not capture the noise, drift, missing readings, unusual operating conditions and complex failure patterns found in real industrial sensors.
- **No confirmed physical sensor integration:** The discussed implementation uses database-fed telemetry and simulation. A production connection to PLCs, SCADA, IoT gateways or actual CNC sensor hardware has not been established in the recorded project history.
- **Ground-truth dependence:** Supervised failure prediction needs reliable failure labels and timestamps. Synthetic or incomplete labels can make evaluation look better or worse than real-world performance.
- **Domain transfer:** A model trained on one simulated machine profile may not generalise to different machine brands, operating loads, maintenance histories or factory environments.

Machine-learning limitations

- **RUL uncertainty:** Remaining Useful Life is an estimate, not an exact countdown. Its reliability depends on degradation data, failure definitions, maintenance events and operating conditions.
- **Prediction is not certainty:** A 24-hour risk prediction indicates estimated risk within a defined horizon; it does not guarantee that a failure will or will not occur.
- **Model validation required:** Chronological train/validation/test splits, class-imbalance handling, leakage checks, calibration and suitable metrics are necessary before claiming reliable performance.
- **False positives and false negatives:** False alarms can cause unnecessary inspections, while missed failures can lead to downtime or equipment damage. Thresholds must be selected with operational costs in mind.
- **Concept drift:** Machine behaviour can change after repairs, component replacement, changes in load or operating conditions. The model may need monitoring and retraining.

System and operational limitations

- **Simulation is not a real-time guarantee:** A 10-minute scheduled analysis and event-driven processing are design targets. Actual latency and reliability depend on scheduler behaviour, data volume, database performance and application uptime.
- **SSE is one-way streaming:** Server-Sent Events support server-to-browser updates. They do not replace ordinary HTTP requests for client actions, nor provide a bidirectional communication channel like WebSockets.
- **Database scaling:** MySQL is suitable for the project scope, but large telemetry volumes require indexing, retention policies, partitioning/archiving strategy and load testing.
- **Security hardening:** Authentication and RBAC are project features, but production use additionally requires secure password handling, session/token controls, input validation, audit logging, secrets management, HTTPS and security testing.
- **Deployment excluded:** Docker and production deployment were deliberately removed from the requested scope. The documented project is therefore a local-development/GitHub deliverable, not a demonstrated production

deployment.

Evidence and evaluation limitations

- Do not claim industrial accuracy, uptime, cost savings or reduced downtime without measured results from a representative evaluation.
- A working UI or successful API response does not, by itself, prove that the ML model predicts failures accurately.
- The project should report the dataset source, generation assumptions, model version, evaluation period, metrics and known failure cases.

5. What should be demonstrated to make the project defensible

- Show the complete data path: simulator or sensor source → validation → MySQL → feature preparation → inference → prediction/alert storage → SSE dashboard.
- Show the difference between offline model training and online inference.
- Demonstrate the 24-hour target and 10-minute schedule, and explain how event-driven inference is triggered.
- Report classification metrics such as precision, recall, F1-score, PR-AUC and confusion matrix; use time-based splits.
- For anomaly detection, explain the anomaly score, threshold selection and how alerts are reviewed.
- For RUL, document the target definition, assumptions, error metric and uncertainty.
- Demonstrate role restrictions, admin access, and how the system behaves when data is missing or invalid.
- Clearly label simulated readings in the UI and documentation so they are not mistaken for live factory data.

6. Suggested project description for a resume or interview

“Developed a web-based Predictive Maintenance System prototype using Python, MySQL and machine-learning techniques to analyse machine telemetry, identify anomalous behaviour and estimate 24-hour failure risk with a Remaining Useful Life output. Designed the workflow around periodic and event-driven inference, role-based access and live dashboard updates using Server-Sent Events. Built and validated the implementation incrementally, using simulated telemetry for development and documenting the limitations that must be addressed before real-world industrial deployment.”

Use the words **prototype**, **simulated telemetry** and **estimated risk** unless and until the system has been tested with real equipment and representative operational data.

7. Closing record

The project journey shows a developer who established the requirements, challenged design assumptions, selected the technologies, narrowed the delivery scope and progressed through implementation in controlled steps. ChatGPT's role was to support selected parts of that process—discussion, explanation, code assistance, troubleshooting and documentation—while the developer remained responsible for decisions, execution, review and understanding the system.

This record is intended as a transparent project-history document. It should be updated as the implementation evolves, especially when real sensor data, model evaluation results, security testing or deployment are added.